**CCS4202-3 DESIGN & ANALYSIS OF ALGORITHM**

**SEMESTER 2 2025/2026**

**LECTURER: DR. AZIAH BINTI ASMAWI**

**FINAL PROJECT REPORT**

**PROJECT TITLE:**

**Smart EV Route & Charging Optimizer for Malaysia**



| MATRIC NO. | NAME |
|---|---|
| 229222 | TENGKU EYMRAN RADHI PUTRA BIN TENGKU RINALFI PUTRA |
| 229229 | IQMAL BIN ISKANDAR |
| 231294 | MUHAMMAD DHIYAUDDIN BIN MOHD SALEH ALAUDIN |
| 227366 | ARIFF DANIAL BIN ZABANI |



**Case Study: A Hari Raya Journey, Johor Bahru to Penang**

# Table of Contents

## 1. Introduction and Background

The adoption of Electric Vehicles (EVs) in Malaysia has increased significantly in recent years due to several contributing factors, including rising fuel prices, government incentives promoting green technology, the expansion of charging infrastructure, and the availability of more affordable EV models. Despite this rapid growth, the supporting charging network remains unevenly distributed. DC fast chargers are primarily located at selected highway Rest and Service (R&R) areas, charging queues are often unpredictable, and limited battery range continues to be a major concern for drivers undertaking long-distance journeys.

Traditional navigation systems are designed to optimize routes mainly based on travel distance or travel time. While this approach is suitable for conventional petrol vehicles, where refuelling is quick, widely available, and relatively consistent in cost, it is less effective for electric vehicles. EV route planning must consider additional factors such as battery capacity, charging station availability, charging duration, waiting time, charging costs, and the potential impact of frequent fast charging on battery health. As a result, route optimization for EVs requires a more comprehensive decision-making approach that balances energy consumption, travel time, operational cost, and battery preservation.

This report presents the development of a **Smart EV Route and Charging Optimizer**, a system designed to identify efficient travel routes while incorporating charging requirements and multiple optimization objectives. The proposed solution evaluates three different algorithmic approaches to determine the most effective strategy for solving the EV route optimization problem.

The remainder of this report is organized as follows. **Section 2** presents a realistic case study illustrating the challenges of EV route planning in Malaysia. **Section 3** compares the three selected algorithmic approaches and discusses their suitability for the problem. **Section 4** formulates the mathematical model used in the optimization process. **Section 5** describes the system design and implementation. **Section 6** demonstrates the system's output and experimental results. **Section 7** provides the theoretical analysis, including algorithm correctness, time complexity, and space complexity. Finally, **Section 8** concludes the report and discusses possible future improvements.

## 2. Problem Illustration: A Festive Cross-Peninsula Journey

Consider **Aisyah**, who must drive from Johor Bahru (JB) to Penang on the eve of Hari Raya along the North-South Expressway (NSE) during the *balik kampung* exodus - the most demanding annual stress-test of Malaysia's road and charging network. Her vehicle, a BYD Atto 3 Extended Range, defines the hard physical limits of the problem.[1]

### 2.1 Vehicle and Battery Constraints

| Parameter | Value / Assumption |
|---|---|
| Usable battery capacity | 60.48 kWh (LFP) |
| Manufacturer range (WLTP) | ~420 km |
| Realistic highway consumption | ~18 kWh/100 km (festive crawl + air-conditioning) |
| Effective real-world range | ~336 km (100% to 0%) |
| Maximum DC charging power | 80 kW (CCS2), tapering above ~50% SoC |
| Starting State of Charge | 100% (charged overnight at home) |
| Emergency reserve policy | Never below 15% SoC (~9 kWh) |

**Implication:** with ~336 km of range and a 15% reserve, only ~285 km of usable range exists between full charges. The ~700 km corridor therefore requires at least two charging stops, and their placement is non-trivial.

### 2.2 Route Segments

| Segment | Distance | Energy* | Terrain / traffic |
|---|---|---|---|
| JB to Ayer Keroh (Melaka) | ~215 km | ~38.7 kWh | Flat; congestion at Senai-Kulai |
| Ayer Keroh to Tapah (Perak) | ~290 km | ~52.2 kWh | Klang Valley bottleneck; stop-go |
| Tapah to Penang (Juru) | ~195 km | ~35.1 kWh | Rolling terrain near Bukit Berapit |
| **TOTAL** | **~700 km** | **~126 kWh** | **~2.1 battery-equivalents** |

[1] BYD Atto 3 Malaysia specs (49.92 / 60.48 kWh; up to 480 km; CCS2 DC). paultan.org, 2022.

**At ~18 kWh/100 km. The Ayer Keroh to Tapah leg (52.2 kWh) exceeds the usable energy above the 15% reserve (~51 kWh), so a single charge cannot safely bridge it - exactly the constraint a distance-only planner ignores.*

### 2.3 Charging Infrastructure (real NSE deployments)

| R&R Stop | Power | Billing (per min) | Festive constraint |
|---|---|---|---|
| Ayer Keroh | 90 kW DC (2 nozzles) | RM2.20/min | Only 2 nozzles - queues build fast |
| Tapah | 80 & 100 kW DC | RM2.05-2.35/min | Most popular hub; longest queues |
| Seremban / others | Upgraded DC (varies) | Comparable | Lower demand - detour candidate |

Two structural traps: the Atto 3 caps intake at 80 kW, so Tapah's 100 kW port gives no benefit at the highest price; and per-minute billing makes the slow taper above 80% the most expensive energy of the trip.[2][3]

### 2.4 Festive Traffic and Queue Dynamics

During the exodus the NSE carried on the order of **2.3 million vehicles per day**, with volumes spiking over 40% and congestion peaking the day before Hari Raya.[4] This raises per-km consumption (crawling + air-conditioning) and turns charging queues into a dominant, time-dependent cost: a peak arrival can mean 30-60 minutes of waiting before charging even begins.

---

[2] TNB Electron / TNBX DC fast chargers at Tapah, Ayer Keroh & Paka R&R. SoyaCincau & AutoBuzz.my, 2023.

[3] TNB Electron per-minute pricing (from RM2.05/min). SoyaCincau, 2023.

[4] PLUS / Buletin Mutiara: NSE festive traffic (~2.3M vehicles/day; >40% spike). 2026.

### 3. Algorithm Paradigms: Comparative Analysis

Three design techniques were selected to solve the routing-and-charging problem: a Greedy heuristic, exact Dynamic Programming (DP), and a Modified Dijkstra / A* search on a state-augmented graph.

| Dimension | Greedy | Dynamic Programming | A* / Dijkstra |
|---|---|---|---|
| Core idea | Best local choice; no look-ahead | Optimal substructure over (node,SoC) | Cheapest-frontier search with heuristic |
| Battery / SoC | Weak - reacts only when low | Excellent - SoC is explicit state | Good - via SoC-augmented states |
| Queues | Poor - picks before knowing wait | Strong - folded into transitions | Moderate - needs time-dependent edges |
| Distance/time | Over-prioritises it | One of several terms | Native strength |
| Optimality | Sub-optimal, can fail feasibility | Globally optimal (within grid) | Optimal if heuristic admissible |
| Best role | Fast baseline / fallback | Offline optimal benchmark | Deployable real-time engine |

**Strategic framing:** use DP as the optimal benchmark, Greedy as the lower-bound baseline, and A* as the deployable solution, then quantify the time-versus-optimality trade-off between them.

## 4. Mathematical Formulation

Model the corridor as a directed graph G = (V, E): nodes are the origin, destination and candidate charging stations; edges are road legs with a distance and a time-dependent travel time. The state tracked is the pair (node, State-of-Charge).

### 4.1 Objective Function

Minimise total elapsed time over the route and charging plan:

```
minimise  T_total = SUM t_drive(e, arrival)
                  + SUM x_i * [ t_queue(i, arrival) +
t_charge(delta_i, P_eff) ]
```

where x_i in {0,1} selects a stop at station i and delta_i is the energy added there. Charging time is non-linear because effective power tapers above the knee SoC.

### 4.2 State Recurrence (Bellman equation)

```
T*(o, B) = 0                              (start full at origin)

T*(j, b_j) = min over (i, b_i, delta) feasible of {
      T*(i, b_i) + t_queue(i) + t_charge(i, b_i, delta) + t_drive(i,
j) }

subject to:  b_j = b_i + delta - e_ij ,  b_j >= b_min ,  b_i + delta
<= B
```

**Feasibility / pruning rule.** A leg is admissible only if b_i + delta - e_ij >= b_min. Any transition violating this (in particular any that would drive the battery below 0%) is discarded immediately and never enters the search.

This is a variant of the Electric Vehicle Shortest-Path Problem with charging decisions, which is **NP-hard**.[5] Edge cost depends on arrival SoC and arrival time, so plain shortest-path methods do not directly apply.

[5] Pelletier, Jabali & Laporte (2017), 'The electric vehicle routing problem with energy considerations' - NP-hardness of EV routing-with-charging.

## 5. Implementation

The optimizer is implemented as a clean, dependency-free Python package. Each algorithm is a class inheriting a common abstract BaseSolver, so all three take identical inputs and are fully interchangeable. All battery physics live in a single Vehicle class, preventing the algorithms from diverging in their assumptions.

```
EV_Optimizer/
  main.py                          # runs the demo scenarios
  ev_optimizer/
    vehicle.py                     # Vehicle: battery + charging
physics
    network.py                     # Network/Station + sample data
    result.py                      # Result / ChargeStop dataclasses
    demo.py                        # run_demo() reporting harness
    algorithms/
      base.py                      # BaseSolver (abstract) + helpers
      greedy.py                    # GreedySolver
      dynamic_programming.py       # DynamicProgrammingSolver
      astar.py                     # AStarSolver
```

**The single safety invariant.** Every transition in all three algorithms passes through one shared gate, so no returned route can ever strand the driver:

```
def is_safe(self, soc_after_charge_kwh, energy_leg_kwh):
    # admissible only if arrival SoC stays at/above the reserve
    # (reserve_kwh >= 0, so this also forbids dropping below 0%)
    return (soc_after_charge_kwh - energy_leg_kwh) >=
self.v.reserve_kwh
```

Representative core of the A* solver (state = (node, SoC band); two successor types - charge and drive):

```
while pq:
    f, gcur, (node, lvl) = heapq.heappop(pq)
    if node == dest: return reconstruct(...)   # first pop = optimal
    # A) charge: station self-transition to a higher SoC band
    # B) drive: each out-edge, pruned by is_safe(lvl, e)
```

Run with: cd EV_Optimizer && python3 main.py (Python 3 standard library only).

## 6. Program Demo and Output

Three scenarios were executed. The festive-peak and off-peak runs find a feasible optimum; the under-charged start is correctly rejected as infeasible.

### 6.1 Scenario 1 - Festive Peak (start 100%)

| Algorithm | Total time | Route / stops |
|---|---|---|
| Greedy | 12h 06m | via Ipoh; 4 stops |
| **Dynamic Programming** | **11h 51m** | **via Ipoh; 3 stops (optimal)** |
| A* (state-augmented) | 11h 51m | via Ipoh; 3 stops (optimal) |

### 6.2 Scenario 2 - Off-Peak (start 100%)

| Algorithm | Total time | Route / stops |
|---|---|---|
| Greedy | 9h 30m | via Tapah; 4 stops |
| **Dynamic Programming** | **9h 23m** | **via Tapah; 3 stops (optimal)** |
| A* (state-augmented) | 9h 23m | via Tapah; 3 stops (optimal) |

**Key insight.** The optimal route switches from the Ipoh path (festive) to the shorter Tapah path (off-peak) once Tapah's 45-minute queue clears - the optimizer reacts to live conditions, not just distance. DP and A* agree exactly in every scenario, cross-validating optimality, while Greedy is consistently 6-15 minutes slower with an extra stop: the measurable price of no look-ahead.

### 6.3 Scenario 3 - Safety Check (start 70%)

The first 215 km leg alone requires ~96% charge, so all three solvers return INFEASIBLE - the strict battery pruning correctly refuses an unsafe trip rather than returning a dangerous plan.

**7. Theoretical Analysis**

**7.1 Proof of Correctness (Dynamic Programming)**

We prove that `DynamicProgrammingSolver` returns a **time-optimal, battery-feasible** plan, optimal up to the SoC discretisation Δ. Dynamic Programming is chosen as the primary algorithm for the formal proof because its correctness follows cleanly from the optimal-substructure property of the DAG; §1.5 gives the complementary optimality argument for A*.

**Lemma 1 (Conservative quantisation).** `quantise_down(x) = ⌊x/Δ⌋·Δ ≤ x`. Hence the level stored for a state never *overstates* the real battery. Consequently, any leg the algorithm accepts as feasible (`is_safe` true on the quantised level) is feasible on the *true* (≥) battery as well. *No infeasible plan is ever reported feasible.*

**Lemma 2 (Optimal substructure).** Consider an optimal feasible plan π* reaching `(u_k, b)`. Let its last transition leave some predecessor state `(u_j, b_j)` (a drive leg `u_j → u_k`, possibly preceded by a charge at `u_j`), with `j < k` because the graph is a DAG. Then the prefix of π* up to `(u_j, b_j)` is itself an optimal plan for `(u_j, b_j)`. *Proof:* the total time is the sum of the prefix time and the (fixed) transition time `queue + charge + drive`; if a cheaper prefix to `(u_j,b_j)` existed, substituting it would yield a cheaper plan to `(u_k,b)`, contradicting the optimality of π*.

**Loop Invariant `I(k)`.** Immediately after the outer loop has finished processing topological node `u_k`, for every node `u_i` with `i ≤ k` and every level `b`, `T_best[(u_i, b)] = OPT(u_i, b)`.

1. Base I(0): only T_best[(start,b0)] = 0; matches OPT.
2. Step: assuming I(k-1), every predecessor of u_k is fully processed (DAG order), so all (charge target, edge) transitions into u_k were relaxed. By Lemma 2 and the inductive hypothesis the minimum candidate equals OPT(u_k,b); every written value corresponds to a feasible plan, so T_best = OPT. Thus I(k) holds.
3. Termination: the state space V*S is finite. Conclusion: T_best[(dest,.)] = OPT(dest,.), and by Lemma 1 the reconstructed plan is truly feasible.

**A* optimality (complement).** The free-flow heuristic is admissible (ignores queue/charge, uses max speed) and consistent (it is itself a shortest free-flow time), so the first pop of the destination is optimal - which is why DP and A* return identical answers.

### 7.2 Time Complexity (V nodes, E edges, S SoC levels)

| Algorithm | Best | Average | Worst |
|---|---|---|---|
| Greedy | O(V + E) | O(E log V) | O(E log V) |
| Dynamic Programming | Th((V+E)S^2) | Th((V+E)S^2) | Th((V+E)S^2) |
| A* / Dijkstra (augmented) | O((V+S)logVS) | <= worst (pruned) | O((SE+VS^2)logVS) |

DP edge-relaxation work sums to S^2 * SUM(outdeg) = S^2 * E, giving a tight bound that grows **quadratically with battery resolution S** (the curse of dimensionality). A*'s augmented edge count is S*E + V*S^2; its admissible heuristic yields far better average-case expansion. Greedy is independent of S - it never enumerates SoC levels.

### 7.3 Space Complexity

| Algorithm | Space | Explanation |
|---|---|---|
| Greedy | O(V + E) | Path, frontier, heuristic map; no SoC grid (independent of S) |
| Dynamic Programming | O(V * S) | Value + back-pointer tables over (node, level) states |
| A* / Dijkstra | O(SE + VS^2) | State maps O(VS); priority queue up to O(augmented edges) |

## 8. Findings and Conclusion

Long-distance EV travel in Malaysia is governed by constraints conventional navigation does not represent: a finite, non-linearly-rechargeable battery; sparse, heterogeneous, capacity-limited chargers; per-minute billing that penalises high SoC; and stochastic, time-dependent queues. Finding a good route is therefore a multi-objective, resource-constrained, NP-hard optimisation problem, not a shortest-path lookup.

Our three-paradigm study yields a clear recommendation: **Dynamic Programming** as the provably-optimal benchmark, **Greedy** as a fast baseline that exposes the cost of no look-ahead, and **A*** as the deployable engine - optimal yet efficient. The agreement of DP and A* across all scenarios validates the optimality claim, and the festive-to-off-peak route switch demonstrates genuine sensitivity to real-world conditions. The project thus turns a single Hari Raya road trip into a rigorous algorithmic problem and solves it three ways.

## References and Data Sources


Buletin Mutiara. (2026). *PLUS anticipates major Raya traffic volume on North–South Expressway.* Accessed via https://www.buletinmutiara.com/plus-expects-surge-in-raya-traffic-on-north-south-expressway/

DistanceBetween2.com. (n.d.). *Geographic distance between Penang and Johor Bahru.* Retrieved July 2, 2026, from https://distancebetween2.com/penang/johor_bahru

Malay Mail. (2023). *TNBX deploys complimentary 90kW DC fast charging at Ayer Keroh and Paka.* https://www.malaymail.com/news/malaysia/2023/01/20/tnbx-offers-free-90kw-ev-dc-fast-chargers-at-ayer-keroh-and-paka-rr/51329

Paul Tan's Automotive News. (2022). *Malaysian launch of BYD Atto 3 EV – dual battery options, 480 km range.* https://paultan.org/2022/12/08/byd-atto-3-ev-officially-launched-in-malaysia-49-92-or-60-48-kwh-up-to-480-km-range-from-rm147800/

Pelletier, S., Jabali, O., & Laporte, G. (2016). Goods distribution with electric vehicles: Review and research perspectives. *Transportation Science, 50*(1), 3–22. https://doi.org/10.1287/trsc.2015.0646

SoyaCincau. (2023a). *TNBX activates 180kW DC fast charging hub at Tapah R&R.* https://soyacincau.com/2023/02/16/tnbx-tnb-electron-ev-dc-fast-charging-at-tapah-rnr-northbound-180kw/

SoyaCincau. (2023b). *Highway DC charging ends free period, transition to time-based billing.* https://soyacincau.com/2023/03/01/tnb-electron-ev-dc-fast-chargers-at-highway-rrs-no-longer-free-priced-from-rm2-05-per-minute/